

■ LEVEL 14 • TREES

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Parent-child relationships. DFS explores branches; BFS explores levels.
Reference Examples	Binary Tree Traversals • Maximum Depth • Diameter • Level Order • Path Sum • Lowest Common Ancestor
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 14 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Maximum Depth of Binary Tree	Tree DFS	#104
2	Invert Binary Tree	Tree DFS	#226
3	Diameter of Binary Tree	Tree DFS + Height	#543
4	Balanced Binary Tree	Tree DFS	#110
5	Path Sum	Tree DFS	#112
6	Binary Tree Level Order Traversal	BFS + Queue	#102
7	Maximum Depth of Binary Tree	Tree DFS	104
8	Invert Binary Tree	Tree DFS	226
9	Diameter of Binary Tree	Postorder DFS	543
10	Binary Tree Level Order Traversal	BFS + Queue	102

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Maximum Depth of Binary Tree

Pattern: Tree DFS

LeetCode: #104

■ Input	binary tree root
■ Output	maximum depth
■ Constraints	binary tree
■ Main Idea / Pattern	Depth is $1 + \max(\text{left depth}, \text{right depth})$.
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(h)$

■ Optimal C++ Code

```
int maxDepth(TreeNode* root) {  
    if (!root) return 0;  
    return 1 + max(maxDepth(root->left),  
                  maxDepth(root->right));  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Invert Binary Tree

Pattern: Tree DFS

LeetCode: #226

■ Input	binary tree root
■ Output	mirrored binary tree
■ Constraints	binary tree
■ Main Idea / Pattern	Swap left and right children at every node recursively.
■■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
TreeNode* invertTree(TreeNode* root) {  
    if (!root) return nullptr;  
    swap(root->left, root->right);  
    invertTree(root->left);  
    invertTree(root->right);  
    return root;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Diameter of Binary Tree

Pattern: Tree DFS + Height

LeetCode: #543

■ Input	binary tree root
■ Output	length of longest path between two nodes
■ Constraints	binary tree
■ Main Idea / Pattern	At each node, diameter through it is left height + right height.
■■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
int diameter = 0;

int height(TreeNode* root) {
    if (!root) return 0;

    int left = height(root->left);
    int right = height(root->right);

    diameter = max(diameter, left + right);
    return 1 + max(left, right);
}

int diameterOfBinaryTree(TreeNode* root) {
    height(root);
    return diameter;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Balanced Binary Tree

Pattern: Tree DFS

LeetCode: #110

■ Input	binary tree root
■ Output	whether tree height difference is at most 1 at every node
■ Constraints	binary tree
■ Main Idea / Pattern	Return height, or -1 if a subtree is unbalanced. Stop early.
■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
int check(TreeNode* root) {
    if (!root) return 0;

    int left = check(root->left);
    if (left == -1) return -1;

    int right = check(root->right);
    if (right == -1) return -1;

    if (abs(left - right) > 1)
        return -1;

    return 1 + max(left, right);
}

bool isBalanced(TreeNode* root) {
    return check(root) != -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Path Sum

Pattern: Tree DFS

LeetCode: #112

■ Input	binary tree and targetSum
■ Output	whether root-to-leaf path sums to target
■ Constraints	binary tree
■ Main Idea / Pattern	Subtract node value while descending. At a leaf, check whether the remaining sum is zero.
■■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
bool hasPathSum(TreeNode* root, int targetSum) {  
    if (!root) return false;  
    if (!root->left && !root->right)  
        return root->val == targetSum;  
    return hasPathSum(root->left, targetSum - root->val) ||  
           hasPathSum(root->right, targetSum - root->val);  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Binary Tree Level Order Traversal

Pattern: BFS + Queue

LeetCode: #102

■ Input	binary tree root
■ Output	values grouped by levels
■ Constraints	binary tree
■ Main Idea / Pattern	Use a queue. Process exactly the current queue size for one level.
■■ Time Complexity	O(n)
■ Space Complexity	O(w)

■ Optimal C++ Code

```
vector<vector<int>> levelOrder(TreeNode* root) {  
    if (!root) return {};  
    vector<vector<int>> ans;  
    queue<TreeNode*> q;  
    q.push(root);  
    while (!q.empty()) {  
        int sz = q.size();  
        vector<int> level;  
        while (sz-->0) {  
            TreeNode* node = q.front();  
            q.pop();  
            level.push_back(node->val);  
            if (node->left) q.push(node->left);  
            if (node->right) q.push(node->right);  
        }  
        ans.push_back(level);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Maximum Depth of Binary Tree

Pattern: Tree DFS

LeetCode: 104

Input	root
Output	maximum depth
Constraints	0 <= nodes <= 1e4
Main Idea / Pattern	Depth is 1 plus the larger depth of the two child subtrees.
Time Complexity	O(n)
Space Complexity	O(h)

Optimal C++ Code

```
class Solution {
public:
    int maxDepth(TreeNode* root) {
        if (!root) return 0;
        return 1 + max(maxDepth(root->left), maxDepth(root->right));
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Invert Binary Tree

Pattern: Tree DFS

LeetCode: 226

Input	root
Output	mirror/inverted tree
Constraints	0 <= nodes <= 100
Main Idea / Pattern	Recursively swap left and right children at every node.
Time Complexity	O(n)
Space Complexity	O(h)

Optimal C++ Code

```
class Solution {
public:
    TreeNode* invertTree(TreeNode* root) {
        if (!root) return nullptr;
        swap(root->left, root->right);
        invertTree(root->left);
        invertTree(root->right);
        return root;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Diameter of Binary Tree

Pattern: Postorder DFS

LeetCode: 543

Input	root
Output	length of the longest path between two nodes
Constraints	0 <= nodes <= 1e4
Main Idea / Pattern	At each node, combine left and right subtree heights to form a path through the node.
Time Complexity	O(n)
Space Complexity	O(h)

Optimal C++ Code

```
class Solution {
    int ans = 0;

    int height(TreeNode* root) {
        if (!root) return 0;

        int left = height(root->left);
        int right = height(root->right);

        ans = max(ans, left + right);

        return 1 + max(left, right);
    }

public:
    int diameterOfBinaryTree(TreeNode* root) {
        height(root);
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Binary Tree Level Order Traversal

Pattern: BFS + Queue

LeetCode: 102

Input	root
Output	values grouped by tree level
Constraints	0 <= nodes <= 2000
Main Idea / Pattern	Use a queue and process exactly its current size before moving to the next level.
Time Complexity	O(n)
Space Complexity	O(w)

Optimal C++ Code

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (!root) return {};
        vector<vector<int>> ans;
        queue<TreeNode*> q;
        q.push(root);
        while (!q.empty()) {
            int sz = q.size();
            vector<int> level;
            while (sz-- > 0) {
                TreeNode* node = q.front();
                q.pop();
                level.push_back(node->val);
                if (node->left) q.push(node->left);
                if (node->right) q.push(node->right);
            }
            ans.push_back(level);
        }
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.